Integración de React en el proyecto Tacos Joya de Cerén: componentes, manejo de estado y consumo de una API externa

Diego Armando Henríquez Fuentes HF261938

Josué Israel Sandoval Alfaro SA260082

Carlos Alexander Rosales Ascencio RA262936

José Ricardo Sandoval Hernández SH262272

Alexander Josué Menjívar MG202784

Nicolas Alejandro Trigueros Molina TM253158

Sistema Computacional

Universidad Don Bosco Virtual

DAW901: Desarrollo de Aplicaciones Web con Software Interpretados en el Cliente

Alexander Alberto Siguenza Campos

8 de noviembre de 2026

Entrega del Foro 2

Etapa 3 Segundo avance

Repositorio

https://github.com/Jexxe0012/tacos-joya-de-ceren

Video de demostración

https://youtu.be/EclpMeqNtr8

Aplicación web

https://jexxe0012.github.io/tacos-joya-de-ceren/

Qué incluye este avance

Este avance cubre los cuatro requerimientos de la actividad: la integración del framework React en los módulos de la aplicación, el consumo de la API externa Open-Meteo con datos reales, los módulos secundarios implementados y funcionales, y el manejo de estado con componentes reutilizables. La investigación sobre el framework aplicado al proyecto comienza en la siguiente página.

Integración de React en el proyecto Tacos Joya de Cerén: componentes, manejo de estado y consumo de una API externa

Hasta el avance anterior, la aplicación de Tacos Joya de Cerén estaba hecha con páginas HTML independientes y JavaScript sin framework. Cada pantalla actualizaba el DOM a mano y repetía la lógica de sesión y de validación. En este avance integramos React para organizar la aplicación en componentes, centralizar el estado y reutilizar piezas entre módulos, y agregamos una API externa con datos reales. Este documento explica cómo aplicamos el framework en el proyecto.

Framework elegido: React

React es una biblioteca de JavaScript para construir interfaces a partir de componentes: funciones que reciben datos y devuelven lo que se muestra en pantalla. Cuando los datos cambian, React vuelve a dibujar solo lo necesario (React, s. f.-a, s. f.-e). La actividad permitía elegir entre AngularJS y React. Escogimos React porque AngularJS dejó de recibir soporte oficial en enero de 2022 (Thompson, 2022) y porque React nos permitía migrar un módulo a la vez, sin reescribir todo de golpe.

Junto con React usamos Vite para el servidor de desarrollo y la compilación (Vite, s. f.), y React Router para la navegación. Las dependencias se instalan con npm y sus versiones quedan fijadas en package-lock.json: React 19.3.1, React Router 7.18.5 y Vite 8.3.5.

Integración del framework en el proyecto

La aplicación pasó de un HTML por módulo a una sola página. El archivo index.html solo tiene un elemento con el id raiz, y src/main.jsx monta ahí el componente App, que envuelve todo con HashRouter y con el proveedor de sesión. Usamos HashRouter porque la aplicación se publica en GitHub Pages, que solo sirve archivos estáticos: la ruta viaja después del símbolo # y

no se envía al servidor (React Router, s. f.), así que recargar en cualquier pantalla no produce un error 404.

Las rutas se declaran en un solo archivo. Los componentes RutaProtegida y RutaPublica deciden quién puede entrar: sin sesión se va al login, y un empleado que intenta abrir una pantalla de administrador vuelve a su inicio (ver Figura 1). Las pantallas más pesadas se cargan con lazy, de modo que su código solo se descarga cuando se abren (React, s. f.-d).

Figura 1

Rutas protegidas por sesión y por rol en src/app/Rutas.jsx

```
<Route element={<RutaProtegida />}>
  <Route element={<AppLayout />}>
    <Route path="/recetas" element={<Recetas />} />
    <Route path="/inventario" element={<Inventario />} />
    <Route path="/mermas" element={<Mermas />} />
    <Route path="/turnos" element={<Turnos />} />
    <Route path="/ventas" element={<VentasPage />} />

    {/* Solo administrador. */}
    <Route element={<RutaProtegida rol={ROL_ADMIN} />}>
      <Route path="/dashboard" element={<Dashboard />} />
      <Route path="/reportes" element={<ReportesPage />} />
      <Route path="/abastecimientos" element={<AbastecimientosPage />} />
    </Route>
  </Route>
</Route>
```

Nota. Fragmento del repositorio del proyecto.

Los módulos principales (autenticación, Recetas, Ventas e Inventario) quedaron como componentes de React. Las librerías del avance anterior se adaptaron: Chart.js se conecta a su lienzo con useRef y se destruye al salir de la pantalla, SweetAlert2 se usa desde un servicio común de diálogos y jsPDF se carga solo al descargar un reporte.

Manejo de estado y componentes reutilizables

Componentes reutilizables

En lugar de repetir el mismo marcado en varias páginas, dividimos la interfaz en componentes con una sola responsabilidad que reciben datos por props (React, s. f.-e). La Tabla 1 resume los compartidos entre módulos.

Tabla 1

Componentes reutilizables del proyecto

Componente	Qué hace	Dónde se usa
AppLayout, Sidebar y Header	Estructura común: menú según el rol, barra superior y contenido de la ruta activa.	Todas las pantallas con sesión
Modal y CapaModal	Ventanas para fichas y formularios.	Recetas, Inventario y Turnos
GraficaBarras	Gráfica de barras con Chart.js y una tabla con los mismos datos.	Dashboard, Ventas y Abastecimiento
ClimaOperativo	Pronóstico de Open-Meteo con estados de carga, error y reintento.	Reportes y Abastecimiento
RutaProtegida y RutaPublica	Control de acceso por sesión y rol.	Todas las rutas

Nota. Elaboración propia a partir del código del repositorio.

Manejo de estado

Organizamos el estado en tres niveles. Lo que solo usa una pantalla, como los filtros de Recetas, se guarda con useState, y la lista visible se calcula con useMemo a partir de esos filtros. La sesión, que necesitan casi todas las pantallas, se comparte con Context: AuthProvider la guarda y cualquier componente la obtiene con el hook useAuth, sin pasarla por props (React, s. f.-b).

El tercer nivel son dos hooks propios para reutilizar lógica con estado (React, s. f.-c). useColeccion entrega los registros de una colección guardada y vuelve a dibujar la pantalla cuando alguien escribe en ella, desde la misma pestaña o desde otra (ver Figura 2). Antes, cada

módulo tenía que actualizar la vista a mano después de guardar. useFormulario maneja los valores, los errores de validación y el envío de cada formulario, y bloquea el botón mientras se guarda para evitar registros duplicados.

Figura 2

Suscripción a cambios en src/hooks/useColeccion.js

```
useEffect(() => {
  recargar();

  const cancelar = db.suscribir(nombre, recargar);

  const alCambiarOtraPestana = evento => {
    if (evento.key === nombre || evento.key === null) recargar();
  };
  window.addEventListener("storage", alCambiarOtraPestana);

  return () => {
    cancelar();
    window.removeEventListener("storage", alCambiarOtraPestana);
  };
}, [nombre, recargar]);
```

Nota. Fragmento del repositorio del proyecto, sin comentarios internos. La función de retorno cancela las suscripciones al desmontar el componente.

Consumo de la API externa Open-Meteo

Integramos Open-Meteo, un servicio de pronóstico del clima que responde en JSON a partir de coordenadas (Open-Meteo, s. f.). La aplicación consulta los próximos siete días para la ubicación del restaurante y muestra temperatura mínima, máxima y probabilidad de lluvia en Reportes y Abastecimiento. Elegimos el clima porque ayuda a planificar compras y turnos en un negocio de comida; aun así, la demanda se sigue calculando con las ventas registradas y la pantalla lo aclara.

Figura 3

Consulta a Open-Meteo en src/services/climaService.js

```
const url = new URL("https://api.open-meteo.com/v1/forecast");
url.search = new URLSearchParams({
  latitude: ubicacionRestaurante.latitud,
  longitude: ubicacionRestaurante.longitud,
  daily:
    "temperature_2m_max,temperature_2m_min,precipitation_probability_max",
  timezone: "America/El_Salvador",
  forecast_days: "7"
});
const limite = AbortSignal.timeout(12000);
```

Nota. Fragmento del repositorio del proyecto.

El servicio limita la petición a 12 segundos, revisa que la respuesta traiga todos los campos esperados, descarta valores inválidos y guarda el resultado 15 minutos para no repetir la consulta en cada cambio de pantalla. El componente ClimaOperativo hace la consulta dentro de useEffect, la forma que React propone para sincronizarse con un sistema externo (React, s. f.-f), y la cancela con un AbortController si el usuario sale de la pantalla (MDN Web Docs, s. f.). Muestra un mensaje mientras carga, un aviso si el proveedor falla y un botón para reintentar; si la API no responde, el resto del módulo sigue funcionando.

Módulos secundarios implementados

Además de los módulos principales, este avance deja funcionando en React los módulos secundarios de la Tabla 2. Todos leen y guardan datos con el hook useColeccion, por lo que se actualizan entre sí: por ejemplo, una merma registrada descuenta la existencia en Inventario y aparece en Reportes sin recargar la página.

Tabla 2

Módulos secundarios del avance

Módulo	Acceso	Función
Mermas	Ambos roles	Registra pérdidas, descuenta inventario y crea el movimiento correspondiente.

Módulo	Acceso	Función
Turnos y solicitudes	Ambos roles	Turnos semanales con revisión de cruces, merma rápida y solicitudes de días libres.
Dashboard	Administrador	Ventas de la semana, consumo esperado, productos críticos y alertas.
Reportes	Administrador	Filtros por período, receta y producto, gráficas, clima y descarga en PDF.
Abastecimiento	Administrador	Simulación de consumo, cobertura y recompra, con el pronóstico del clima.

Nota. Elaboración propia a partir del código del repositorio.

Conclusión del avance

Con este avance la aplicación funciona como una sola aplicación React. Las rutas protegidas reemplazaron la lógica de sesión repetida en cada página, Context y los hooks propios centralizaron el estado, los componentes compartidos evitaron duplicar marcado y Open-Meteo agregó datos reales sin afectar la operación cuando el servicio falla. Los datos siguen guardándose en el navegador, como en el prototipo anterior. Esta base es la que se completará para la entrega final del proyecto.

Referencias

MDN Web Docs. (s. f.). *AbortController*. Mozilla. Recuperado el 8 de noviembre de 2026, de https://developer.mozilla.org/es/docs/Web/API/AbortController

Open-Meteo. (s. f.). *Weather forecast API*. Recuperado el 8 de noviembre de 2026, de https://open-meteo.com/en/docs

React. (s. f.-a). *Quick start*. Meta Open Source. Recuperado el 8 de noviembre de 2026, de https://react.dev/learn

React. (s. f.-b). *Passing data deeply with context*. Meta Open Source. Recuperado el 8 de noviembre de 2026, de https://react.dev/learn/passing-data-deeply-with-context

React. (s. f.-c). *Reusing logic with custom hooks*. Meta Open Source. Recuperado el 8 de noviembre de 2026, de https://react.dev/learn/reusing-logic-with-custom-hooks

React. (s. f.-d). *lazy*. Meta Open Source. Recuperado el 8 de noviembre de 2026, de https://react.dev/reference/react/lazy

React. (s. f.-e). *Thinking in React*. Meta Open Source. Recuperado el 8 de noviembre de 2026, de https://react.dev/learn/thinking-in-react

React. (s. f.-f). *Synchronizing with effects*. Meta Open Source. Recuperado el 8 de noviembre de 2026, de https://react.dev/learn/synchronizing-with-effects

React Router. (s. f.). *HashRouter*. Recuperado el 8 de noviembre de 2026, de https://reactrouter.com/api/declarative-routers/HashRouter

Thompson, M. (2022, 11 de enero). *Discontinued long term support for AngularJS*. Angular Blog. https://blog.angular.io/discontinued-long-term-support-for-angularjs-cc066b82e65a

Vite. (s. f.). *Getting started*. Recuperado el 8 de noviembre de 2026, de https://vite.dev/guide/